Mapa de soluciones · 5 ejercicios

ERP Frescura Total · rama `test`, commit `4a738a5` · Dónde está cada parte de la solución y cómo recorrerla.

Cómo usar este mapa en IntelliJ

- **Ctrl + Shift + N**: abrir un archivo por su nombre (por ejemplo `CustomerCache`).
- **Ctrl + G**: ir a un número de línea.
- **Ctrl + Shift + F**: buscar `EJERCICIO 3` en todo el proyecto. Cada cambio está marcado con ese comentario.
- **Ctrl + clic** sobre un método: saltar a su definición y seguir el flujo.

Todas las rutas de clases empiezan en `src/main/java/com/frescuratotal/erp/`. Los números de línea son los del commit `4a738a5`.

Resumen

#	Ejercicio	Menú	Archivo principal	Prueba rápida
1	Caché de clientes (Singleton)	Sales → 6	`sales/model/CustomerService.java`	`CacheTest` (necesita BD)
2	Reporte CSV de compras	Inventory → 10	`inventory/model/PurchaseReportService.java`	Menú → abre el .csv
3	Tarifas con Strategy	Sales → 3	`sales/model/DeliveryService.java`	`StrategyTest` (sin BD)
4	Importar clientes desde .txt	Sales → 7	`sales/model/CustomerImportService.java`	`clientes_importar.txt`
5	Estadísticas con Streams	Sales → 8	`sales/model/CommercialStatsService.java`	Menú + `UPDATE` del .sql

Archivos fuera del código Java

Archivo	Para qué
`docs/practica_examen.sql`	Agrega el estado **Devuelta (id 6)**, necesario para el ejercicio 5, y trae consultas para probar los ejercicios 1, 2, 4 y 5. **Ejecútalo primero.**
`clientes_importar.txt` (raíz)	Archivo de ejemplo del ejercicio 4, con 5 clientes y una línea vacía.
`compras_AAAA-MM-DD.csv` (raíz)	Lo **genera** el ejercicio 2 al usar el menú. No está en el repositorio.

Archivos existentes que se modificaron

Archivo	Cambio	Ejercicio
`sales/view/SalesView.java`	Opciones 6, 7 y 8 en el menú (Back pasa a ser la 9) · `readDeliveryType` · `readCustomerId` · `showImportErrors` · `showCommercialStats`	1, 3, 4, 5
`sales/controller/SalesController.java`	Los `case` 6 a 9 · `while (option != 9)` · tres métodos nuevos · modalidad de entrega en `newSale`	1, 3, 4, 5
`sales/model/SaleService.java`	Atributos `deliveryService` y `customerService` · `confirmSale(sale, delivery)` · `findCustomerById`	1, 3
`sales/model/SaleStatus.java`	Nueva constante `RETURNED(6)`	5
`inventory/view/InventoryView.java`	Opción 10 en el menú (Back pasa a ser la 11) · `showPurchaseReport`	2
`inventory/controller/InventoryController.java`	`case 10` y `case 11` · `while (option != 11)`	2

Ejercicio 1 · Caché de clientes con Singleton

Menú: 2. Sales → 6. Find customer by ID (cache)

Flujo (sigue las flechas con Ctrl + clic)

VISTA
`SalesView readCustomerId()` L170
→
CONTROLADOR
`SalesController findCustomerById()` L119
→
SERVICIO
`SaleService findCustomerById()` L48
→
DECIDE
`CustomerService findById()` L33
→
MEMORIA / BD
`CustomerCache` o `CustomerDao`

Dónde está cada requerimiento

Requerimiento	Archivo	Dónde
1. Almacenar en memoria por ID	`sales/model/CustomerCache.java`	`L30 Map<Integer, Customer>`
2. Primera consulta: BD → guardar → devolver	`sales/model/CustomerService.java`	`L45–56`
3. Consultas posteriores sin SQL	`sales/model/CustomerService.java`	`L35–42`
4. Instancia única (Singleton)	`CustomerCache.java`: instancia, constructor privado y `getInstance()`	`L25 · L33 · L37`
4. VentaService y ClienteService comparten caché	`CustomerService.java` (pide `getInstance()`) y `SaleService.java` (usa CustomerService)	`CS L21 · SS L31`
5. Decisión encapsulada (no en menú ni vista)	`CustomerService.findById`. El controlador solo recibe un Optional	`L33`
6. No existe → informar	`SalesController.findCustomerById`	`L125`
6. No guardar nulos ni inválidos	`CustomerCache.put` (null o id ≤ 0) y `ifPresent`	`L55–60 · CS L52`

Cómo probarlo

- **Prueba completa:** clic derecho en `sales/model/CacheTest.java` → Run. Debe imprimir `[SQL]`, `[CACHE]`, `[SQL]`, `[CACHE]`, `[SQL]` + not found, y luego `Same cache instance? true`.
- **Desde el menú:** Sales → 6 con el mismo ID dos veces. La primera vez sale `[SQL]` y la segunda `[CACHE]`. Si sales al menú principal y vuelves a entrar, sigue saliendo `[CACHE]`, porque la caché es `static`.
- Si los IDs 5 u 8 no existen en tu BD, revísalos con la consulta del .sql y cámbialos en `CacheTest`.

Ejercicio 2 · Reporte diario de compras en CSV

Menú: 1. Inventory → 10. Generate daily purchase report (CSV)

Flujo

CONTROLADOR	SERVICIO	DAO	ARCHIVO	VISTA
`InventoryController` `case 10` L104	`PurchaseReportService` `generateDailyReport()` L40	`PurchaseReportDaoImpl` `findPurchasesByDate()` L33	`compras_AAAA-MM-DD.csv`	`InventoryView` `showPurchaseReport()` L58

Equivalencia que tienes que explicar: el ERP no tiene una tabla de compras. Una **compra** es la **recepción de un lote** (`lotes_inventario`). Por eso NumeroCompra = `numero_lote`, el proveedor sale de un JOIN con `proveedores` y MontoTotal = `cantidad_inicial × precio_compra`. Está documentado en `inventory/model/DailyPurchase.java`.

Dónde está cada requerimiento

Requerimiento	Archivo	Dónde
1. Opción en el menú, sin pedir datos	`InventoryView.showMenu` · `InventoryController`	`IV L29 · IC L104`
2. Fecha automática con Java	`PurchaseReportService.generateDailyReport`: `LocalDate.now()`	`L42`
3. Solo las compras de hoy (id, proveedor, total)	`inventory/model/PurchaseReportDaoImpl.java`: SQL con `DATE(fecha_ingreso) = ?`	`L22–30`
4. Nombre `compras_AAAA-MM-DD.csv` en la raíz	`PurchaseReportService`: `Paths.get("")` y `resolve(...)`	`L42 · L50`
5. Delimitador ; y cabecera exacta	Constantes `HEADER` y `DELIMITER`	`L26–27`
5. Punto decimal, sin $ ni miles	`formatAmount`: `setScale(2).toPlainString()`	`L79–81`
Escritura del archivo	`generateReport`: try-with-resources con `BufferedWriter`	`L54–65`
6. Día sin compras → solo cabecera y aviso	El archivo se escribe siempre. El aviso está en `InventoryView.showPurchaseReport`	`IV L58–66`

Cómo probarlo

- **Con compras hoy:** Inventory → 3 (registra uno o dos lotes) → Inventory → 10. El .csv aparece en la raíz del proyecto, junto a `pom.xml`. Ábrelo con el Bloc de notas, no con Excel.
- **Sin compras:** prueba un día sin recepciones (o con la BD recién cargada). El archivo sale solo con la cabecera y se muestra el aviso.
- **Compras de días anteriores:** no aparecen, por el filtro `DATE(...) = hoy`. Puedes compararlo con la consulta de la sección "EJERCICIO 2" en `docs/practica_examen.sql`.

Ejercicio 3 · Tarifas de entrega con Strategy

Menú: 2. Sales → 3. New sale (la pregunta "Delivery type" aparece antes de confirmar)

Flujo



Dónde está cada requerimiento

Requerimiento	Archivo	Dónde
1. Contrato común (interfaz)	`sales/model/DeliveryStrategy.java`: `calculateTotal` y `getDescription`	L16
2. ESTANDAR (0%)	`sales/model/StandardDeliveryStrategy.java`	L14–17
2. URGENTE (12%)	`sales/model/UrgentDeliveryStrategy.java`: constante `1.12`	L17 · L20–25
Modalidad elegida por el usuario	`sales/model/DeliveryType.java` (enum STANDARD, URGENT)	L12
3. Recibir venta y modalidad, elegir e invocar	`sales/model/DeliveryService.java`: `calculateFinalAmount`	L15–28
3. Selección de la estrategia (switch)	`DeliveryService.getStrategy`	L36–44
Redondeo a 2 decimales	`DeliveryService`: `setScale(2, HALF_UP)`	L27
4. Integración: preguntar antes de confirmar	`SalesController.newSale`	L100
4. Integración: aplicar después de los totales	`SaleService.confirmSale`	L111 → L116
Mostrar la descripción al cliente	`SaleService.getDeliveryDescription` → controlador	SS L43
5. El switch no calcula y el % vive en la estrategia	Ningún `case` calcula y el 1.12 solo está en `UrgentDeliveryStrategy`	DS L41–42

Cómo probarlo

- **Los 5 casos, sin BD:** clic derecho en `sales/model/StrategyTest.java` → Run. Las 5 líneas deben terminar en `OK`.
- **Integrado:** haz una venta con STANDARD y otra igual con URGENT. El TOTAL de la segunda debe ser el primero × 1.12, y debajo del recibo aparece la descripción.

Ejercicio 4 · Importación masiva de clientes

Menú: 2. Sales → 7. Import customers from file

Flujo

CONTROLADOR
`SalesController importCustomers()` L132
→
FASE 1: VALIDAR
`CustomerImportService validate()` L88
→
FASE 2: GUARDAR
`importCustomers()` `inTransaction` L76
→
DAO
`CustomerDaoImpl.save` (uno por cliente)
→
ERRORES
`ImportException` → `showImportErrors`

Dónde está cada requerimiento

Requerimiento	Archivo / método	Dónde
1. Opción del menú	`SalesView.showMenu` · `SalesController` case 7	`SC L43`
2. Archivo en la raíz	`CustomerImportService`: `Paths.get("clientes_importar.txt")`	`L29`
4. El archivo existe	`importCustomers`: `Files.exists`	`L58`
3. Cabecera exacta	`validate` (quita el BOM y compara)	`L95–98`
4. Ignorar líneas vacías	`validate`: `isBlank()` → `continue`	`L109`
4. Cinco campos no vacíos	`split(";", -1)` + `length != 5` + campos vacíos	`L116 · L119 · L131`
4. Correo válido	Regex `EMAIL`	`L42 · L137`
4. Solo MAYORISTA o MINORISTA	`toCustomerType` (→ WHOLESALE / RETAIL)	`L166`
4. Documento repetido (archivo / BD)	`Set.add` y `customerDao.findByDocument`	`L148 · L150`
5. Validar todo antes de guardar	Se juntan los errores y se lanzan al final	`L159–161`
5. Sin registros parciales (rollback)	`DatabaseConnection.inTransaction`	`L76`
6. Mensajes exactos	Constantes `MSG_NOT_FOUND` y `MSG_INVALID` · éxito en el controlador	`CIS L34–35 · SC L135`

Cómo probar los 6 casos

Caso	Qué hacer con `clientes_importar.txt`
Todos válidos	Déjalo como está → "Importación completada. Total de clientes registrados: 5"
Línea vacía intermedia	Ya trae una (línea 4) y se ignora
Archivo inexistente	Renómbralo, por ejemplo a `clientes_importar.bak`
Cabecera incorrecta	Cambia `nombre` por `Nombre`
Registro con 4 campos	Borra `;MINORISTA` de una fila
Documento duplicado	Vuelve a importar el mismo archivo. Para repetir desde cero, usa el `DELETE` del .sql

Ejercicio 5 · Estadísticas comerciales con Stream API

Menú: 2. Sales → 8. Generate commercial statistics

Flujo



Equivalencias que tienes que explicar: el ERP no tiene el estado CONFIRMADA. Toma como **confirmada** toda venta guardada que no esté cancelada ni devuelta (`isConfirmed`, L80). **DEVUELTA** es el estado nuevo `RETURNED(6)` en `SaleStatus.java` (L12), y se crea en la BD con `docs/practica_examen.sql`.

Dónde está cada requerimiento

Requerimiento	Archivo / método	Dónde
2. JOIN de ventas, detalles, lote, producto, categoría y cliente	`sales/model/SalesStatsDaoImpl.java`	L22–34
2. Filas convertidas en objetos Java	`SaleStatRow` (record) · `while (rs.next())`	SSR L12 · DAO L47–55
3. Solo ventas confirmadas	`filter(row -> isConfirmed(...))`	L50
3. Agrupar por categoría y sumar	`groupingBy` + `reducing(BigDecimal::add)`	L51–53
3. Categoría con mayor acumulado	`max(Map.Entry.comparingByValue())`	L59–61
4. Filtrar devueltas → nombres → sin duplicados	`filter` → `map` → `distinct` → `toList`	L69–74
5. Sin for ni while en esas operaciones	Todo `calculate()` usa Streams (el `while` del DAO solo lee filas)	L40–77
6. Sin ventas → mensaje	`generate()` devuelve Optional vacío → controlador	CSS L33–35 · SC L148
6. Sin devoluciones → sin errores	Lista vacía; la vista imprime solo el título	SV L197–198
Formato de salida exacto	`SalesView.showCommercialStats`	L186–199

Cómo probarlo

- **Primero** ejecuta `docs/practica_examen.sql`. Sin el estado 6, las devoluciones no existen.
- **Sin devoluciones:** Sales → 8 muestra la categoría top y el título "Clientes con devoluciones:" vacío.
- **Con devoluciones:** en el .sql, ejecuta `UPDATE ventas SET id_estado = 6 WHERE id_venta IN (x, y);` con dos ventas del mismo cliente. Ese cliente aparece **una sola vez**, y esas unidades dejan de sumar en las categorías.
- Para deshacerlo, ejecuta el `UPDATE ... SET id_estado = 1` que está comentado en el mismo .sql.